

DD2352 Algorithms and Complexity
Spring 2026
Lecture 9: Complexity Intro
(Turing Machines and Sorting Lower Bounds)

Per Austrin

2026-03-17

Agenda

Algorithms Recap and Complexity Intro

Turing Machines and Models of Computation

Lower Bound for Comparison-Based Sorting Algorithms

Algorithms Recap (1/3)

- ▶ Greedy
 - ▶ Build solution step by step.
 - ▶ Choose “locally best” option each step.
 - ▶ Main difficulty often to prove correctness.
- ▶ Divide & Conquer
 - ▶ Split problem into *small number of significantly smaller subproblems*.
- ▶ Dynamic Programming
 - ▶ Similar to greedy but instead of predicting best choice in each step, we explore all choices.
 - ▶ Exploit structure of the problem to make sure that only small number of *different* subproblems encountered.
 - ▶ Save computed results to avoid solving same subproblems over and over again.

Algorithms Recap (2/3)

- ▶ Flow Networks and Maximum Flow
 - ▶ Models flow of some divisible “fluid” through a graph
 - ▶ Interested in maximum flow from s to t subject to throughput capacities.
- ▶ Flow Applications
 - ▶ Many problems can be solved by transforming them to maxflow
 - ▶ Minimum Edge/Vertex Cuts
 - ▶ Bipartite Matching
 - ▶ Disjoint Paths
 - ▶ ...and many more

Algorithms Recap (3/3)

- ▶ Randomized Algorithms

- ▶ Monte Carlo algorithms: may sometimes fail to find the right answer.
- ▶ Las Vegas algorithms: may sometimes run much longer than expected.

Computational Complexity

So far in the course we have been looking at ways of designing efficient algorithms for different problems.

For the problems we encountered so far we managed to do this.

Is this always possible, are there no limitations to what we can do with algorithms if we are sufficiently clever?

Or are there problems that *require* unreasonable amounts of time to solve?

This is the basic question we will now start to explore.

Complexity of an Algorithm vs a Problem

So far in the course we have been analyzing the time complexity of specific algorithms.

We are now shifting focus to the time complexity of a computational problem: the best possible time complexity of *any* algorithm that solves the problem.

Hardness is Hard

In most cases, proving a **time complexity lower bound**, that a problem *requires* a high time complexity, is very difficult.

In general, proving something does not exist can be challenging, and the space of all possible algorithms is very complicated.

For some very complex problems, lower bounds can be shown, but for most natural problems, we cannot currently prove any non-trivial lower bounds.

For the last 50+ years, theoretical computer scientists have tackled this brick wall in many different ways.

Today we take a brief look at one such way: **restricted models of computation**.

But first, we have to talk about **models of computation**.

Agenda

Algorithms Recap and Complexity Intro

Turing Machines and Models of Computation

Lower Bound for Comparison-Based Sorting Algorithms

What is an algorithm?

We have an informal intuition that “an algorithm” is something we can run on a computer.

But:

to prove mathematical statements about computation

we need a *mathematical model of computation*

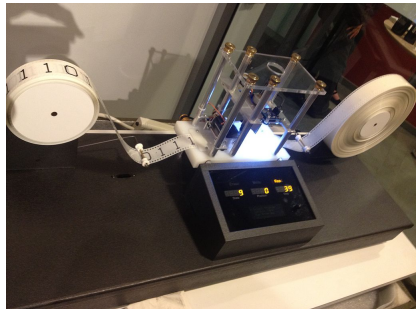
(a precise mathematical definition of what an algorithm is)

Turing Machines

Enter **Turing machines**

Introduced by British mathematician Alan Turing in 1936 as part of solution to the so-called “**Entscheidungsproblem**”.

Short description:
simplified computer which has an infinite memory tape that it can move back and forth on and modify, based on what is on the tape and what state the machine is in.



(Picture by Gabrielf on Wikipedia, cc by-sa)

Turing Machine Examples

We walk through the examples “repeat 0 1” and “equal strings” on <http://turingmachine.io/>.

Picture on the previous slide notwithstanding, Turing machines are primarily a mathematical construction, not physical devices!

Components of a Turing Machine

In more detail, a Turing machine (TM) consists of:

- ▶ A finite set of **states**

One of the states is the **initial state**

(when the TM starts, it is in this state)

Some of the states are **accepting states**

(if the TM ends in one of these states, it says “yes”)

- ▶ **Transition rules** that determine what to do based on current state and content of memory tape at current position.
 - ▶ What (if anything) will be written to the memory tape
 - ▶ Whether the read/write head should move one step left or right
 - ▶ What the new state should be
- ▶ An **alphabet**, the set of symbols that can be stored on the memory tape

The Church-Turing Hypothesis

Are Turing machines a good formal model?

You might object:

- ▶ They have no abstraction, and are painful to work with for everything but the simplest example programs.
- ▶ Not immediately clear that they have any useful bearing on real-world computation.

One explanation for the importance of Turing machines is the

Church-Turing Thesis:

Anything which can be computed by any automatic method, can also be computed by a Turing machine.

Not a mathematical theorem but a scientific hypothesis about how the physical world works.

The RAM Model (1/2)

Another important model of computation is the RAM
(“RAM = Random Access Machine”)

Models the way modern computers operate, with registers and memory cells that can be immediately accessed.

Essentially a form of **Assembly language**, comes with small set of basic instructions e.g.:

- ▶ copying data between registers and memory cells
- ▶ perform simple operations (add, subtract, compare, conditional jumps, etc)

When we design algorithms and write pseudo code, we typically implicitly work in the RAM model.

RAM Model, Example

Consider the following program.

(Operates on first three memory cells denoted `mem[0]`, `mem[1]`, `mem[2]` and a single register denoted `reg0`.)

```
1: cmp mem[0] with mem[1]
2: jump to 6 if "<="
3: reg0 <- mem[0]
4: mem[0] <- mem[1]
5: mem[1] <- reg0
6: cmp mem[1] with mem[2]
7: jump to 11 if "<="
8: reg0 <- mem[1]
9: mem[1] <- mem[2]
10: mem[2] <- reg0
11: cmp mem[0] with mem[1]
12: jump to 16 if "<="
13: reg0 <- mem[0]
14: mem[0] <- mem[1]
15: mem[1] <- reg0
16: stop
```

Example run:

Memory:

5	7	2
---	---	---

Register:

0

RAM Model, Example

Consider the following program.

(Operates on first three memory cells denoted `mem[0]`, `mem[1]`, `mem[2]` and a single register denoted `reg0`.)

```
1: cmp mem[0] with mem[1]
2: jump to 6 if "<="
3: reg0 <- mem[0]
4: mem[0] <- mem[1]
5: mem[1] <- reg0
6: cmp mem[1] with mem[2]
7: jump to 11 if "<="
8: reg0 <- mem[1]
9: mem[1] <- mem[2]
10: mem[2] <- reg0
11: cmp mem[0] with mem[1]
12: jump to 16 if "<="
13: reg0 <- mem[0]
14: mem[0] <- mem[1]
15: mem[1] <- reg0
16: stop
```

Example run:

Memory:

5	7	2
---	---	---

Register:

0

After running program:

2	5	7
---	---	---

(done step-by-step in lecture)

What does the program do
in general?

The RAM Model (2/2)

Several variations on RAM model, e.g.:

- ▶ **Word RAM model**: registers and memory cells have a limited word size of w bits; adds bit parallelism
- ▶ **Parallel RAM (PRAM)** model: models a machine with large number of processors/cores.
- ▶ Probabilistic RAM model: adds access to randomness

Previously we have talked about unit cost and bit cost complexity

- ▶ unit cost complexity $\approx$ complexity in the basic RAM model
- ▶ bit cost complexity $\approx$ complexity in the Word RAM model

RAM Model References

The course book does not discuss the RAM model.

A good reference is the first two sections of the paper [Sorting and searching on the word RAM](#) by Torben Hagerup, which defines the Word RAM model.

Some lecture slides from other universities, discussing the RAM model in a bit more detail:

<https://www.cse.cuhk.edu.hk/~taoyf/course/comp3506/lec/ram.pdf>
or

http://people.cs.bris.ac.uk/~konrad/courses/2019_2020_COMS10007/slides/03-Model-of-Computation-short.pdf (pages 9-14)

Other Models of Computation

Turing Machines and the RAM Model are examples of **abstract machines**, or models of computation.

(The two most important ones!)

Many other models of computation exist

(λ -calculus, external memory model, Boolean circuits, straight-line programs, cellular automata, minecraft, etc)

Church-Turing Thesis: “anything these other models can do, a Turing machine can also do.”

If a model can also do everything that a Turing machine can, then it is called a **Turing-complete** model of computation.

Agenda

Algorithms Recap and Complexity Intro

Turing Machines and Models of Computation

Lower Bound for Comparison-Based Sorting Algorithms

Complexity of sorting

We know many algorithms for sorting a list of objects:

- ▶ Bubble sort, insertion sort, etc have complexity $\Theta(n^2)$
- ▶ Merge sort, heap-sort, ... have complexity $\Theta(n \log n)$

What does this tell us about the *complexity of the sorting problem*?

Each algorithm gives an **upper bound** on complexity of sorting:
 $O(n \log n)$ due to the existence of algorithms which are that fast

But this says nothing about possible existence of faster algorithms.
Could we solve it in $O(n \log \log n)$ or $O(n)$ or even $\Theta(\sqrt{n})$?

A trivial **lower bound**: any algorithm needs to look at the entire input and thus have complexity $\Omega(n)$ – means $\Theta(\sqrt{n})$ is impossible

Goal: learn the precise complexity of sorting – close the gap between the $\Omega(n)$ lower bound and $O(n \log n)$ upper bound

Restricted Models of Computation

It is very difficult to reason mathematically about all possible algorithms.

Natural to restrict attention to algorithms that are of some limited form.

Examples:

- ▶ Simple computational models that are not Turing-Complete
E.g.: finite automata such as a **deterministic finite automaton** (DFA) – like Turing machine but no memory
- ▶ Limit what the algorithm is allowed to do in some way
E.g. **sorting algorithms that only use comparisons to make decisions.**

Comparison-Based Sorting

Suppose we have a sorting algorithm which given input array $a_1, \dots, a_n$ only makes decisions based on comparing two elements (checking if $a_i < a_j$)

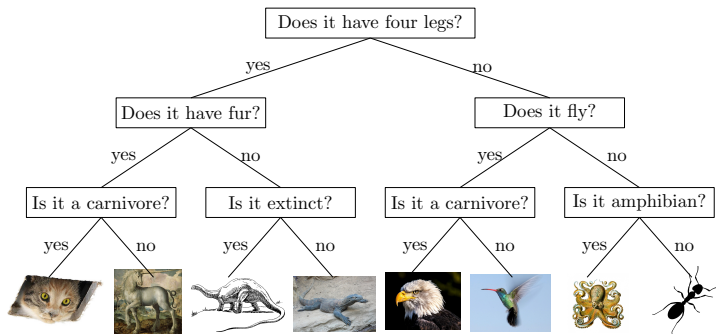
We will prove a **lower bound**: any such algorithm must make $\Omega(n \log n)$ comparisons (and thus take $\Omega(n \log n)$ time)

We already have an **upper bound**:
we know algorithms that take $O(n \log n)$ time (e.g. Merge Sort)

So we conclude that the time complexity of comparison-based sorting is $\Theta(n \log n)$.

Aside: animal questions game

3 questions game: identify an animal with 3 questions.



With 3 questions we can distinguish between 8 types of animals.

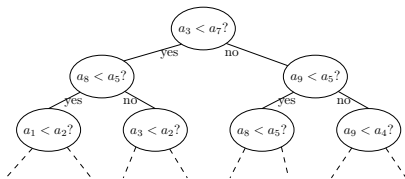
In general with q questions, we can identify 2^q types of animals.

So if there are A many different animals, we need $\lceil \log_2(A) \rceil$ questions to distinguish them.

Sorting Lower Bound

We can view the progress of a comparison-based sorting algorithm in exactly the same way:

- ▶ The correct reordering of the input array is an “animal” we are trying to identify.
- ▶ Each comparison the algorithm makes is a question.



There are in total $n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$ permutations on n elements that we have to distinguish.

So algorithm needs to make at least $\log_2(n!)$ comparisons.

Sorting Lower Bound, continued

How does $\log_2(n!)$ behave?

- ▶ **Method 1: direct calculation.** We have

$$\begin{aligned}\log_2(n!) &= \sum_{i=1}^n \log_2(i) \geq \frac{n}{2} \log_2(n/2) \\ &= \frac{1}{2}(n \log_2(n) - n) = \Omega(n \log n)\end{aligned}$$

- ▶ **Method 2: use Stirling's approximation.**

Stirling's formula says that $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ so we have

$$\log_2(n!) \approx \frac{1}{2} \log_2(2\pi n) + n \log_2(n) - n \log_2(e) = \Theta(n \log n)$$

Bottom line: *comparison-based sorting requires making at least $\log_2(n!) = \Omega(n \log n)$ comparisons.*

Is Comparison-Based Sorting a Natural Restriction?

Assuming that the algorithm only operates by comparing elements seems like a natural restriction.

If the input elements are some arbitrary objects then this is really all the algorithm can do.

But if the input elements are e.g. small integers then an algorithm could do other things! Better algorithms actually exist in this case.

- ▶ **Counting sort:**

- if a_i 's are integers between 0 and n we can sort in $O(n)$ time.

- ▶ **Faster sorting of integers:** can sort n word-sized integers in $O(n \log \log n)$ time in general

Upcoming week in the course:

- ▶ Wednesday/Thursday 10-12: Exercise 6
Randomized Algorithms
- ▶ Monday 15-17: Lecture 10
(Un)decidability and Reductions
- ▶ Tuesday 15-17: Lecture 11
P, NP, and NP-completeness